

For the three grayscale test images test1.img, test2.img and test3.img (in .img format) provided in eLC, show the result of the following thresholding algorithms:

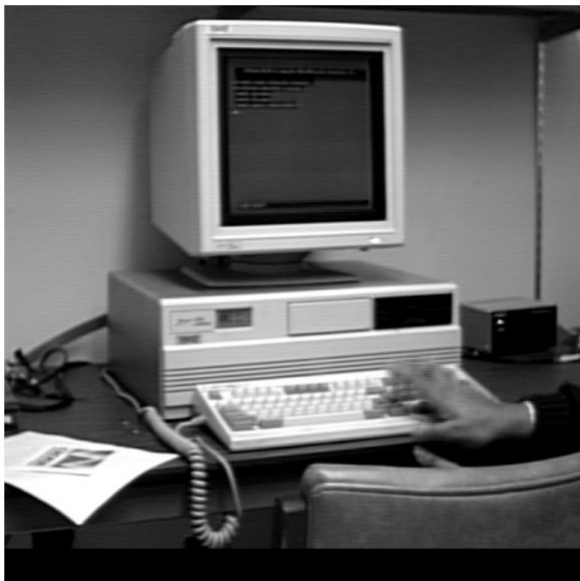
1. Thresholding using peakiness detection. Explain clearly the evaluation criterion used to select the pair of grayscale peaks and the intervening valley.

I used the local_minmax function to find all the local peaks and valleys in the image histogram. Then, I used the FOM_Calculator to evaluate each pair of peaks and the valley between them by calculating the multiplication version of the Figure of Merit (FOM) mentioned in the Computer Vision Class.

$$\frac{\alpha \cdot H(P_i) \cdot H(P_j) \cdot \beta \cdot (P_j - P_i)}{(\gamma \cdot (H(V_k) + 1)) \cdot \delta \cdot \left(\left| \frac{P_i + P_j}{2} - V_k \right| + 1 \right)}$$

This FOM considers the height and separation of the peaks as well as the depth and centrality of the valley. The pair with the highest FOM indicates the best separation between regions in the image, so I chose the corresponding valley as the threshold. This threshold is then used to segment the image, and its value is shown in the title of each binary image:

Original Image



Peakiness Thresholding (Threshold = 99)



Original Image



Peakiness Thresholding (Threshold = 155)



Original Image



Peakiness Thresholding (Threshold = 171)



As you see, different threshold values have been selected for three images by the algorithm. A lower threshold classifies more mid-range gray values as white, whereas a higher threshold emphasizes only the brightest regions as white. The third image seems to have the lowest contrast of objects between the three images, so we have fewer distinct peaks, and because the image is mostly lighter, the threshold is higher than others.

Computer Vision HW#3

Nima Darbandi

811296007

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  input_file = '3_Thresholding/test2.img'
5  output_file = '3_Thresholding/test1B.img'
6
7  # -----
8  # ----- FUNCTIONS -----
9
10 # Function to calculate the histogram
11 def get_histogram(image_data):
12     hist,_ = np.histogram(image_data, bins=256, range=(0, 255))
13     return hist
14
15 # Function to show the histogram
16 def show_hist(hist):
17     plt.bar(range(256), hist)
18     plt.title('Histogram')
19     plt.xlabel('Pixel Value')
20     plt.ylabel('Frequency')
21     plt.show()
22
23 # Function to calculate the local minima (valeys) and maxima (peaks) in the histogram
24 def local_minmax(hist):
25     min = []
26     max = []
27     for i in range(1, len(hist) - 1):
28         if hist[i] > hist[i - 1] and hist[i] > hist[i + 1]:
29             max.append(i)
30         elif hist[i] < hist[i - 1] and hist[i] < hist[i + 1]:
31             min.append(i)
32     return min, max #Valeys and Peaks
33
34 # Function to calculate the Figure of Merit (FOM)
35 def FOM_Calculator(Pi, Pj, Vk, hist, alpha=1, beta=1, gamma=1, delta=1):
36     FOM = alpha * hist[Pi] * hist[Pj] * beta * (Pj-Pi) / ( (gamma*(hist[Vk]+1)) * delta*(abs(((Pi+Pj)/2)-Vk)+1))
37     return FOM
38
39 # Function to show the images for debugging
40 def show_images(image_data, final_image, title):
41     fig, axs = plt.subplots(1, 2, figsize=(10, 5))
42
43     axs[0].imshow(image_data, cmap='gray')
44     axs[0].set_title('Original Image')
45     axs[0].axis('off')
46
47     axs[1].imshow(final_image, cmap='gray')
48
49     axs[1].set_title(title)
50     axs[1].axis('off')
```

Computer Vision HW#3

Nima Darbandi

811296007

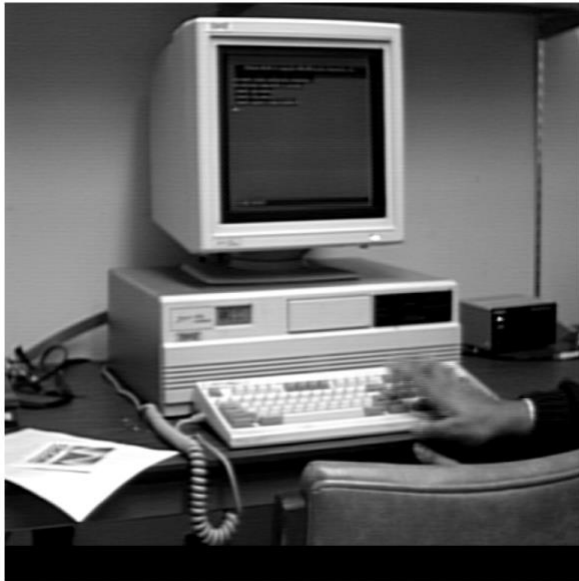
```
51
52     # axs[2].imshow(binary_128, cmap='gray')
53     # axs[2].set_title('Threshold = 128')
54     # axs[2].axis('off')
55
56     plt.show()
57 # -----
58 # -----main-----
59
60 # Read the raw binary data
61 image_data = np.fromfile(input_file, dtype=np.uint8)
62
63
64 # Reshape the image data into a 512x512 array
65 image_data = image_data[512:]
66 image_data = image_data.reshape((512, 512))
67
68
69 # Calculate the histogram of the image data
70 hist = get_histogram(image_data)
71
72 # get the list of valeys (local minima) and peaks (local maxima) in the histogram
73 lmin, lmax = local_minmax(hist)
74
75 # Find the optimum Figure of Merit (FOM) and the corresponding threshold value
76 optimum_FOM = 0
77 threshold_value = None
78
79 for i in range(len(lmax)):
80     for j in range(i+1, len(lmax)):
81         for k in range(len(lmin)):
82             FOM = FOM_Calculator(lmax[i], lmax[j], lmin[k], hist)
83             if FOM > optimum_FOM:
84                 optimum_FOM = FOM
85                 threshold_value = lmin[k]
86
87
88 # Thresholding using Peakiness detection
89 binary_image = np.where(image_data > threshold_value, 255, 0).astype(np.uint8)
90
91
92 # Display the binary image peakiness thresholding
93 title = f'Peakiness Thresholding (Threshold = {threshold_value})'
94 show_images(image_data, binary_image, title)
95
96
97
98 # # Save image
99 # binary_image.tofile(output_file)
```

2. Iterative thresholding.

Iterative Threshold Selection:

- make an initial guess and select an initial value for the threshold $T = T_{init}$
 T_{init} is equal to the average of all the gray levels in the image
- Partition the image into two classes, R_1 and R_2 , based on T
- Compute the mean μ_1 and μ_2 of R_1 and R_2
- Update $T = (\mu_1 + \mu_2) / 2$
- Repeat steps 2-4 until T doesn't change between successive iterations
-

Original Image



Iterative Thresholding (Threshold = 105)



Original Image



Iterative Thresholding (Threshold = 106)



Original Image



Iterative Thresholding (Threshold = 136)



Here, the algorithm starts with the average grayscale value, splits the image into two classes, computes each class's mean, and updates the threshold until it becomes stable. Because we are using averaging, the thresholding can show a greater distinction between objects in images. So, when we compare it to the previous algorithm, it shows a better separation of objects.

Computer Vision HW#3

Nima Darbandi

811296007

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  input_file = '3_Thresholding/test3.img'
5  output_file = '3_Thresholding/combB.img'
6
7  # -----
8  # ----- FUNCTIONS -----
9
10 # Function to calculate the histogram
11 def get_histogram(image_data):
12     hist, _ = np.histogram(image_data, bins=256, range=(0, 255))
13     return hist
14
15 # Function to show the histogram
16 def show_hist(hist):
17     plt.bar(range(256), hist)
18     plt.title('Histogram')
19     plt.xlabel('Pixel Value')
20     plt.ylabel('Frequency')
21     plt.show()
22
23 # Function to perform iterative thresholding with a maximum number of iterations
24 def iterative_thresholding(image_data, max_iterations=1000):
25     T = np.mean(image_data)
26
27     for _ in range(max_iterations):
28         # Partition the image into two classes based on threshold T
29         R1 = image_data[image_data <= T]
30         R2 = image_data[image_data > T]
31
32         u1 = np.mean(R1)
33         u2 = np.mean(R2)
34
35         new_T = (u1 + u2) / 2
36
37         if np.abs(new_T - T) == 0: # Convergence condition
38             break
39
40         T = new_T
41
42     return T
43
44
45 # Function to show the images for debugging
46 def show_images(image_data, final_image, title):
47     fig, axes = plt.subplots(1, 2, figsize=(10, 5))
48
49     axes[0].imshow(image_data, cmap='gray')
50     axes[0].set_title('Original Image')
51     axes[0].axis('off')
```

Computer Vision HW#3

Nima Darbandi

811296007

```
51     axs[0].axis('off')
52
53     axs[1].imshow(final_image, cmap='gray')
54
55     axs[1].set_title(title)
56     axs[1].axis('off')
57
58     # axs[2].imshow(binary_128, cmap='gray')
59     # axs[2].set_title('Threshold = 128')
60     # axs[2].axis('off')
61
62     plt.show()
63
64     # -----
65     # -----main-----
66     # Read the raw binary data
67     image_data = np.fromfile(input_file, dtype=np.uint8)
68
69     # Reshape the image data into a 512x512 array
70     image_data = image_data[512:]
71     image_data = image_data.reshape((512, 512))
72
73     # Calculate histogram of the image data
74     hist = get_histogram(image_data)
75     threshold_value = iterative_thresholding(image_data)
76
77     # Generate the binary image using the threshold value obtained from iterative thresholding
78     binary_image = np.where(image_data > threshold_value, 255, 0).astype(np.uint8)
79
80     # # Thresholding using 128 Threshold
81     # binary_image128 = np.where(image_data > 128, 255, 0).astype(np.uint8)
82
83     # Display the binary image peakiness thresholding
84     title = f'Iterative Thresholding (Threshold = {threshold_value:.0f})'
85     show_images(image_data, binary_image, title)
```

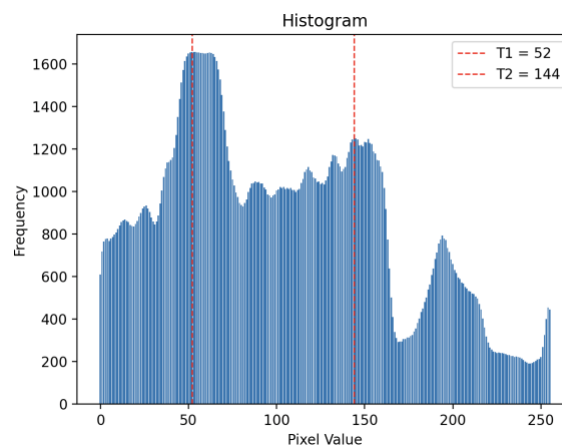
3. Dual thresholding with region growing. Explain how you would choose the two threshold values automatically from the histogram. Explain clearly the logic behind the heuristic(s) for the selection of the two threshold values.

Before threshold selection, I smoothed the histogram because the histogram peaks showed playing a crucial role in threshold selection.

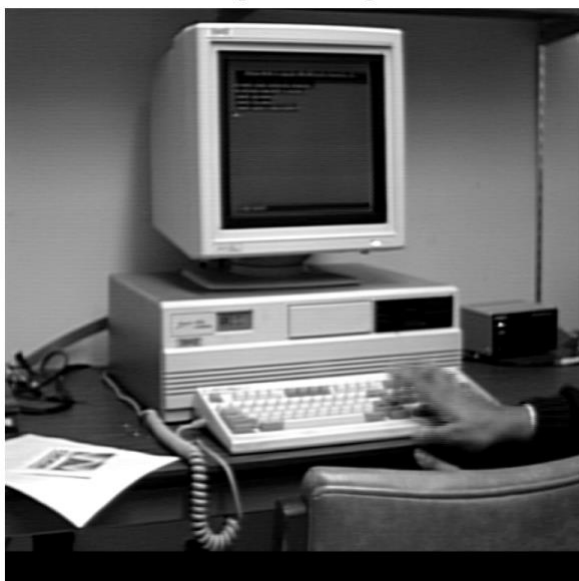
First, I used the `substitute_outliers_with_max` function to get the upper 10% of the histogram values. Then, I substituted higher bound outliers with max of inliers because I needed all 256 grayscale values and couldn't eliminate any of the outliers. Then, I smoothed the histogram with a Gaussian smoother with a kernel size of 9 to optimize the threshold-finding process.

With the smoothed histogram, the algorithm scans through each grayscale value and, within a fixed window (50 pixels in this case), checks if the value is the maximum in that range. If it is, it's marked as a candidate peak. This windowed approach helps ensure that only significant, well-separated peaks are considered, rather than local fluctuations which are not in our interest.

Once the candidate peaks are identified, the smallest candidate is chosen as T1 and the largest as T2. The window size is a hyperparameter and we can tune it by user.

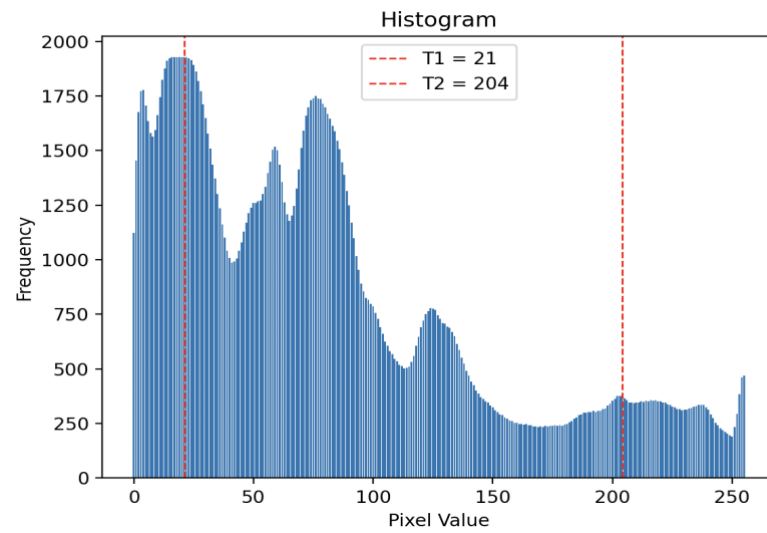


Original Image



Dual Thresholding (T1 = 52, T2 = 144)

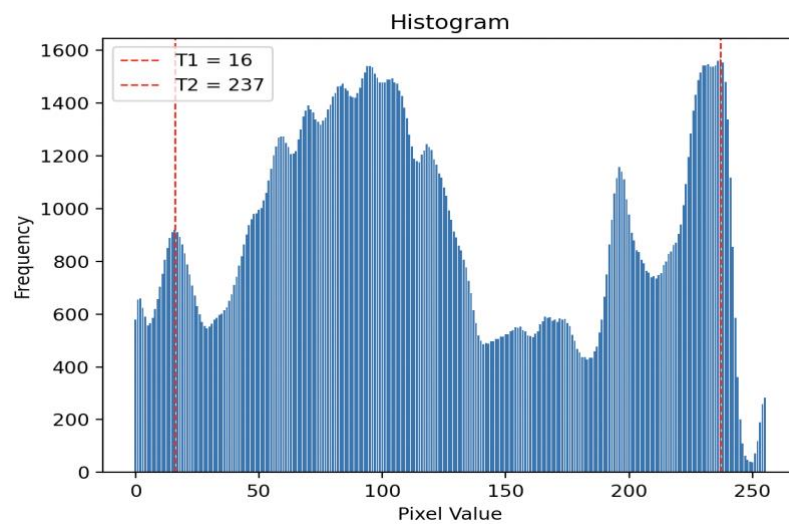




Original Image



Dual Thresholding (T1 = 21, T2 = 204)



Original Image



Dual Thresholding ($T1 = 16$, $T2 = 237$)



I spent a significant amount of time on this algorithm because I couldn't achieve optimal object separation. Although the method suggests choosing two conservative threshold values, I tried several different initial threshold scenarios, but nothing changed. The main issue was the region-growing step, which did not produce good results for the last image. I included the histogram plot and the threshold results selected by my FOM function to illustrate the steps I took. It seems the iterative thresholding is doing best for me between these three algorithms.

Computer Vision HW#3

Nima Darbandi

811296007

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4
5  input_file = '3_Thresholding/test3.img'
6  output_file = '3_Thresholding/combB.img'
7
8
9  # -----
10 # -----FUNCTIONS-----
11 # Function to calculate the histogram
12 def get_histogram(image_data):
13     hist, _ = np.histogram(image_data, bins=256, range=(0, 255))
14     return hist
15
16 # Function to show the histogram
17 def show_hist(hist, T1, T2):
18     plt.figure(figsize=(7, 3))
19     plt.bar(range(256), hist)
20     plt.axvline(x=T1, color='red', linestyle='--', linewidth=1, label=f'T1 = {T1}')
21     plt.axvline(x=T2, color='red', linestyle='--', linewidth=1, label=f'T2 = {T2}')
22     plt.title('Histogram')
23     plt.xlabel('Pixel Value')
24     plt.ylabel('Frequency')
25     plt.legend()
26     plt.show()
27
28 # Function to substitute outliers with other values to smooth the histogram
29 def substitute_outliers_with_max(data):
30     lower_bound = np.percentile(data, 10)
31     upper_bound = np.percentile(data, 90)
32
33     inliers = data[(data >= lower_bound) & (data <= upper_bound)]
34     mean_val = np.mean(inliers)
35
36     data_sub = np.copy(data)
37
38     # Replace outlier values with the mean of inliers and 0
39     data_sub[data < lower_bound] = 0
40     data_sub[data > upper_bound] = np.max(inliers)
41
42     return data_sub
43
44 # Function to smooth the histogram using a Gaussian kernel
45 def histogram_smoother(hist, kernel_size=9, sigma=2.0):
46
47     if kernel_size % 2 == 0:
48         kernel_size += 1
49
50     half_size = kernel_size // 2
```

Computer Vision HW#3

Nima Darbandi

811296007

```
50     half_size = kernel_size // 2
51     x = np.arange(-half_size, half_size + 1)
52
53     kernel = np.exp(-x**2 / (2 * sigma**2))
54
55     kernel /= kernel.sum()
56     smoothed_hist = np.convolve(hist, kernel, mode='same')
57
58     return smoothed_hist
59
60
61 # Function to select the peak pair for dual thresholding
62 # This function uses a 50 pixel window to find the most distant and dominant peaks in the histogram
63 ✓ def select_peak_pair(hist, window=50):
64
65     n = len(hist)
66     candidates = []
67
68     for i in range(n):
69         left = max(0, i - window)
70         right = min(n, i + window + 1)
71         # Check if the current value is the maximum within the window
72         if hist[i] == np.max(hist[left:right]):
73             if hist[i] > hist[i - 1] and hist[i] > hist[i + 1]: # Ensure it's a peak not a max
74                 candidates.append(i)
75
76     if len(candidates) < 2:
77         return None, None # Not enough peaks found
78
79     peak1 = min(candidates) # smallest peak (T1)
80     peak2 = max(candidates) # largest peak (T2)
81
82     return peak1, peak2
83
84
85 # Function to show the images
86 ✓ def show_images(image_data, final_image, title):
87     fig, axs = plt.subplots(1, 2, figsize=(10, 5))
88
89     axs[0].imshow(image_data, cmap='gray')
90     axs[0].set_title('Original Image')
91     axs[0].axis('off')
92
93     axs[1].imshow(final_image, cmap='gray')
94
95     axs[1].set_title(title)
96     axs[1].axis('off')
97
98     # axs[2].imshow(binary_128, cmap='gray')
```

Computer Vision HW#3

Nima Darbandi

811296007

```
98     # axs[2].imshow(binary_128, cmap='gray')
99     # axs[2].set_title('Threshold = 128')
100    # axs[2].axis('off')
101
102    plt.show()
103
104    # -----
105    # -----main-----
106    # Read the raw binary data
107    image_data = np.fromfile(input_file, dtype=np.uint8)
108
109    # Reshape the image data into a 512x512 array
110    image_data = image_data[512:]
111    image_data = image_data.reshape((512, 512))
112
113    # Calculate histogram of the image data
114    hist = get_histogram(image_data)
115
116    hist_no_outlier = substitute_outliers_with_max(hist)
117    hist = histogram_smoother(hist_no_outlier, kernel_size=9, sigma=2.0)
118    optimal_T1, optimal_T2 = select_peak_pair(hist, window=30)
119    #print("\n\n T1, T2 =", (optimal_T1, optimal_T2))
120
121    # Show the smoothed histogram
122    show_hist(hist, optimal_T1, optimal_T2)
123
124    # Double thresholding
125    double_thresh = np.ones_like(image_data, dtype=np.uint8) # set all as R2 (1)
126    double_thresh[image_data <= optimal_T1] = 0 # R1 (0)
127    double_thresh[image_data >= optimal_T2] = 2 # R3 (2)
128
129    # Neighbors offsets
130    neighbors_8 = [(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0), (1, 1)]
131    neighbors_4 = [(-1, 0), (0, -1), (0, 1), (1, 0)]
132
133
134    # Visit each pixel in class R2 if it has a neighbor in class R1, then reassign pixels to class R1
135    changed = True
136    height, width = double_thresh.shape
137    while changed:
138        changed = False
139        # For each pixel in R2, check if it touches an R1 pixel
140        for x in range(height):
141            for y in range(width):
142                if double_thresh[x, y] == 1: # R2
143                    # Check neighbors
144                    for dx, dy in neighbors_8:
145                        nx = x + dx
146                        ny = y + dy
```

Computer Vision HW#3

Nima Darbandi

811296007

```
145         nx = x + dx
146         ny = y + dy
147         if 0 <= nx < height and 0 <= ny < width:
148             if double_thresh[nx, ny] == 0: # neighbor is R1
149                 double_thresh[x, y] = 0 # reassign to R1
150                 changed = True
151                 break
152
153     # Remaining R2 --> R3
154     double_thresh[double_thresh == 1] = 2
155
156
157
158
159     doubleT_image = np.where(double_thresh == 0, 0, 255).astype(np.uint8)
160     #binary_128 = np.where(image_data > 128, 255, 0).astype(np.uint8)
161     title = f'Dual Thresholding (T1 = {optimal_T1}, T2 = {optimal_T2})'
162     show_images(image_data, doubleT_image, title)
```